

your project sucks

why foss needs gardeners, not influencers

10 minute read

2025-11-15

<new | old>

~ note

I've created a website that condenses my arguments into a small page: [stopslopware.net](#). It serves as an educational resource, and a means of spreading the word about the “stop slopware” movement. If you'd like to invite a project author to consider de-slopifying their work, consider sharing a link to the site.

- the weeds are spreading
- substance vs. performance
- indicators of pretend projects
- what good foss looks like
- the argument for gatekeeping
- keep the commons clean
- errata

A specter is haunting the open–source community: the specter of *slop*.

It's impossible to escape. Visit any mainstream software or programming oriented community, and you'll see it completely overwhelmed by one garbage post after the other.

A lot has been written about why AI generated code is problematic, often non–functional, and unethical. Basically, why we call it ***slop***. I'm not here to contribute to that corpus. I think that task is much better suited to programmers and thinkers far more experienced than I.

The problem I want to talk about today isn't exactly caused by AI and its consequences; but much like all forms of online spam, LLMs have certainly made the problem much worse.

I'm talking about dishonest, low–effort **slop projects**: how they're poisoning the culture of open software collaboration, and making the ~~FOSS~~ world more painful for experienced developers, and less accessible for newcomers.

In this article, I'm going to be making an argument for **gatekeeping**. Not people, but *culture*, which is always worth defending. I'm not advocating for elitism, credentialism, or hostility towards beginners.

Instead, I say we should band together and defend the norms, values, quality standards, and our shared understanding of what open–source *is for*. In others words, I argue we should fight to preserve the things that made us fall in love with the ~~FOSS~~ community in the first place.

What I advocate is closer to keeping a garden than guarding a castle; there are weeds that choke out the plants, and if no one pulls them out, **the garden dies**.

the weeds are spreading

The open–source ecosystem has a problem: it's being polluted by **garbage**. To be clear: I'm not talking about “bad code”. I'm not talking about posts by newcomers that don't follow best practices or haven't yet learned the conventions and expectations.

Like any programmer, I’ve written my fair share of shitty code. In fact, I *continue* to do so; most things I write are going to be liquid ass before I get around to polishing and fixing them. That’s expected, it’s part of the process.

But if you’re a novice, you may not have the *skills* necessary to make your bad code better. That’s okay, too. We were all beginners once, and barring newcomers from participating at all just because they can’t meet a certain quality bar is simply cruel and unfair. If we want our community to grow and sustain itself, it’s a given that novices should be *taught* how to participate.

When I talk about “garbage”, I’m not referring to something bereft of skill. I’m talking about projects that are *low–effort*, and *dishonest*. And this isn’t just a personal annoyance; it’s ecosystem pollution.

Slop projects clog up discovery and discussion channels, bury genuinely useful content under piles of noise, dilute the meaning of “open–source project”, and encourage inexperienced devs to mimic bad patterns. It harms everyone, *especially* newcomers. Taking a stand against slop isn’t toxic gatekeeping, it makes our spaces *more* welcoming by creating space for real novices.

substance vs. performance

As alluded to earlier, there is a distinction between “project slop” and AI slop. There may be a some overlap, but the concepts are not identical. In my opinion, while slop projects are *more often than not* majority AI–generated, *they don’t have to be*. Similarly, a project whose author made **heavy** use of AI tooling is not *automatically* slop; it’s just *very likely to be*.

If I had to define what separates slop projects from the rest, I’d say it comes down to their **release–first mentality** (as opposed to the standard “problem–first mentality”). Where “good” projects are usually built because the author needed to solve a problem, slop is built because the author wants attention. Where real projects are refined over time, slop is frontloaded with marketing and quickly abandoned.

There’s a *core dishonesty* to be addressed. Slop **performs** “project-ness” rather than actually *being* a project. They may mimic the aesthetics of one: a code of conduct, issue templates, contribution guidelines, development roadmaps, but they’re missing one crucial component: **substance**.

When you publish something under the *banner* of open–source, you implicitly enter a **stewardship role**. You’re not just shipping files, you’re making a contribution to a *shared commons*. That carries certain responsibilities: clarity about purpose, honesty about limitations, and a basic alignment with the community’s collaborative ethos.

Slop projects violate this social contract. They exploit the aesthetics of open–source without embodying the underlying ethos. The result is something that is not useful, does not solve its stated problem, and is often *knowingly* misrepresented by its authors. Engineering takes a backseat to marketing, and the “open–source” label becomes a kind of free advertising.

These “works” aren’t made to help people or solve problems, and their contribution isn’t a selfless act of giving. This cheapens the ecosystem. It erodes our trust in what it *means* for a project to exist, to be maintained. Slop treats open–source not as a commons to steward, but a stage to perform on.

indicators of pretend projects

It’s quite difficult to derive *exactly* the bounds of project slop. It can take on many different forms, and is constantly evolving, making attempts at defining a taxonomy an exercise in futility.

Instead, I’ve compiled a (subjective) list of general **red flags**: things that tip me off, or at least make me suspicious, that the project I’m looking at is, indeed, garbage.

Before getting into it, I'd like to point out the common thread across all these examples: these are, essentially, beginner's projects **that overstate their own importance**.

A huge disclaimer: there is *absolutely nothing wrong with being new*, nor should we discourage anyone from publishing their learner's projects and asking for feedback.

The problem isn't that these projects are made by inexperienced developers, it's that they're being **fundamentally dishonest about what they're delivering**.

If you wrote a shitty todo list app, by all means, please share that and ask for feedback; just don't promise me a "revolutionary productivity framework that redefines workflows, made with ❤️ by developers for developers".

Similarly, there's no shame in copying something else or reinventing the wheel, (there's really no better way to learn new skills), just don't pretend like you're doing something other than *learning*.

~ note

There may be completely valid, *non-slop* projects that tick a couple of these boxes. **Please treat these as a starting point for further investigation**, not the be-all end-all.

Without further ado, here's *my* list of project slop red flags:

1. The project has a website using the same vibe-coded template as every other slop project.

- You know exactly which one I'm talking about.
- Bonus points if there's so much blur and smooth scrolling that my fairly-modern Android phone can barely render what's *supposed* to be a documentation website at 15 FPS.

2. The only selling point is the open-source label.

- This doesn't include ~~OSS~~ alternatives to proprietary software; rather, software for which open-source solutions *already* exist, but the author can't tell us anything that sets their project apart.
- If "it's open-source!" is the only thing you can say about your work, I'll assume it doesn't have any other substance.

3. Begging for contributions on Day One.

- The pattern I've noticed: serious, well-intentioned developers don't beg for contributors; they let the project speak for itself, to which contribution will flow naturally.¹
- There's nothing wrong with asking for contributors; but there's something about being *begged* to contribute to brand-new weekend projects that puts a sour taste in my mouth.
- I've even had random people reply to (unrelated!) Reddit comments asking me to contribute to their project! Like, what?
- You are the maintainer, and there are no users, like, build something before outsourcing your obligations!

4. Community scaffolding, sans the *community*.

- When there's a large amount of community scaffolding on a project that's very new and doesn't have any actual community activity — code of conduct, issue templates, and at least half the README is a "how to contribute guide".
- The README has more badges, feature lists, and scaffolding than actual technical detail.
- Things that leave me with a feeling the project is more focused on "community growth" than delivering software.

5. Obviously AI-generated announcement post or README.

- This one should go without saying.
- Emoji overuse.
- Way too much bold text per bullet point.
- Light on technical details, and the feature list includes stuff like “lightweight CLI” and “one-click install”.

6. Is related to AI tooling in any way.

- This one may seem a bit unfair; that’s why this list is subjective.
- I’m sure there exists actually good AI tooling, but I’ll be honest, if I see a project whose description involves “LLM” or “MCP” literally anywhere, my *immediate* assumption is that the whole thing is vibe-coded garbage. And frankly, so far, that impulse has been correct.

7. Assumes GitHub is synonymous with `git`.

- Occasionally, I’ll run into projects that bill themselves as “git utilities”, just to find that they only support GitHub through its proprietary API, and make no effort to integrate with... well, `git`.
- When projects claim “`git` integration” and then make no effort to support any provider other than GitHub, it tells me the author doesn’t understand the ecosystem they’re claiming to participate in.²
- If your tool only works on one corporate platform and breaks the moment someone uses Codeberg, you haven’t built a “`git` tool”, you’ve built a GitHub client dressed up as something more serious.

8. Seems more focused on the stack than the result.

- If your README lists “written in Rust for blazingly fast speeds 🚀” *before* even telling me what it does, I’m immediately leaving.
- If the most interesting thing to say about your program is the language it’s written in or the tech stack you used, I’m going to assume there isn’t anything else worth engaging with.

9. Announced before the project even exists.

- When the project has a logo, name, reserved domain, website, roadmap, etc, but *little to no implementation*.
- This tells me that the author is more interested in the image of being a maintainer than in shipping anything real.

what good foss looks like

You may already know the smell of garbage, so let me also share a brief, affirmative vision. As before, these are subject “green flags”, not some authoritative list.

- Small tools with small scopes.
- README is short, accurate, written *after* the fact.
 - + (It should be long only if there’s actual substance to document).
- No community scaffolding until a community actually exists.
- Author clearly uses their own project.
- Website (if any) is functional, plain, documentation-focused.
- No hype, no emoji fireworks.

the argument for gatekeeping

Gatekeeping should not be about skill Level. Good `QSS` can be written by a beginner, and good `QSS` can still be bad code. Similarly, Bad `QSS` can be written by a senior engineer: it could be “good code” that follows “best practices”, but still be a garbage *project*.

Project slop isn’t about junior mistakes, it’s about dishonesty and performance. We shouldn’t care about resume prestige, programming language purity, or whether someone knows the “right” patterns. We care whether the author respects what open source **is**. So gatekeeping, here, is *defending a cultural ethic*, not excluding new people.

Gatekeeping is a responsibility, not a superiority complex. I’m not some `FOSS` titan. I maintain a handful of projects that solve problems for me and

a few other people. I receive the occasional issue, email, and contribution.

That’s enough. Because I care about this community, I have a stake in keeping it healthy. Being a steward of open–source doesn’t require fame or success; the only credential I care about is an indication that you *care* about this ecosystem.

Gatekeeping can be a form of cultural transmission. Healthy communities have boundaries which are meant to *teach* newcomers how to participate. Gatekeeping is how people learn the rules:

- DON’T publish things that solve nothing, UNLESS you’re explicit about it being a learning project.
- DON’T pretend your toy project is something revolutionary.
- DO describe your project with honesty and integrity.
- DON’T expect feedback, but DO expect respect if you receive any.

Gatekeeping is what keeps these norms alive. Otherwise, the culture is diluted by people that don’t actually like software, just the aesthetics of being seen as someone that does.

Protecting the QSS culture is a shared responsibility because QSS is a *commons*. Repositories, discovery platforms, community feeds, these all get destroyed without stewardship. Slop is pollution, and gatekeeping is conservation.

By doing our part to reject low–effort slop from our communities, we protect newcomers from thinking slop is normal & emulating bad patterns, maintainers from drowning in noise, and users from garbage tools with misleading claims.

We can create a culture where small projects are respected, “boring” tools are valued, honesty is rewarded, craftsmanship matters, and developers feel safe to publish imperfect but *real* work.

In other words, we can protect those who contribute in good faith.

keep the commons clean

If you want to publish something unfinished, fine. If you want to learn in public, great. **But don’t lie.** Don’t market a half–finished weekend script as the next revolution. Don’t pretend you’re contributing when you’re really just performing.

Gatekeeping is just calling that out.

Everyone benefits from a culture built on honesty instead of cosmetics, beginners especially. Nothing sabotages new developers faster than the belief that vaporware and empty branding are normal parts of the craft.

If your project has substance, the culture will catch you. If it doesn’t, no amount of emojis or boiler plate can save it.

The fix is simple: make real things.

Or don’t.

Just stop pretending.

errata

1. A friend made an excellent additional point: serious developers sometimes may have already learned that intaking contributions is actually work too, and can be quite difficult considering that you don’t know how well the contributor aligns with your goals; whether they may be taking your project in the wrong direction, etc. So, people contributing to your project isn’t necessarily a 100% benefit anyways. Thus, begging for contributions is (often) another sign of a maintainer with the wrong priorities. ↩

2. It was pointed out to me that the Forgejo and Gitea APIs closely mimic the GitHub API, such that if you point an application to one of those endpoints instead, it's possible that it'll work. I think my greater point still stands, but I'd be remiss not to mention this. ↩

[return to top](#)

© Daniel, 2025

[source available](#)

built with [zona](#) on Dec 21, 2025 @ 14:51 EST